Range Sum of BST (easy)

Problem Statement

Given a Binary Search Tree (BST) and a range defined by two integers, L and R, calculate the sum of all the values of nodes that fall within this range. The node's value is inclusive within the range if and only if L <= node's value <= R.

Examples:

Example 1:

Input:

```
Tree:
   10
  /  \
 5   15
/ \   \
3   7   18
Range: [7, 15]
```

Expected Output: 32

Justification: The values that fall within the range [7, 15] are 7, 10, and 15. Their sum is 7 + 10 + 15 = 32.

Example 2:

Input:

```
Tree:
   20
  /  \
 5   25
/ \
3   10
Range: [3, 10]
```

Expected Output: 18

Justification: The values within the range [3, 10] are 3, 5, and 10. Their sum is 3 + 5 + 10 = 18.

Example 3:

Input:

```
Tree:
   30
     \
     35
    /
   32
Range: [30, 34]
```

Expected Output: 62

Justification: The values within the range [30, 34] are 30 and 32. Their sum is 30 + 32 = 62.

Try it yourself

Try solving this question here:

Java

```java
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {

    public int rangeSumBST(TreeNode root, int L, int R) {
        // ToDo: Write Your Code Here.
        return root.val;
    }
}
```

Test Case | Run Code Result | Stdout

```
[10,5,15,3,7,null,18]
7
15
```

Use Example Testcases | Run | Submit

← Previous

Solution: Minimum Difference Between BST Nodes (Easy)

Next →

Solution: Range Sum Of BST (Easy)

Mark as Completed